SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

I ______HARIHARAN.R(192524003)______________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:HARIHARAN.R

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understan ding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

ANSWER:

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis.

KNOWLEDGE BASE

1. If a patient has Fever AND Cough → Possible Flu
2. If a patient has Fever AND Rash → Possible Measles
3. If a patient has Cough AND Breathlessness → Possible Pneumonia
4. Facts about Patient A:

Fever = True

Cough = True

Breathlessness = True

(a) REPRESENTATION USING PROPOSITIONAL LOGIC

PROPOSITIONAL SYMBOLS

Let:

F = Patient A has Fever

C = Patient A has Cough

R = Patient A has Rash

B = Patient A has Breathlessness

FL = Patient A has Possible Flu

M = Patient A has Possible Measles

P = Patient A has Possible Pneumonia

RULES IN PROPOSITIONAL LOGIC

Rule 1:

Fever AND Cough → Possible Flu

$F \land C \rightarrow FL$

Rule 2:

Fever AND Rash → Possible Measles

$F \land R \rightarrow M$

Rule 3:

Cough AND Breathlessness → Possible Pneumonia

$C \land B \rightarrow P$

FACTS

F = True

C = True

B = True

Therefore, the complete knowledge base is:
(F ∧ C → FL)
∧
(F ∧ R → M)
∧
(C ∧ B → P)
∧
F
∧
C
∧
B

(b) FORWARD CHAINING
DEFINITION
Forward Chaining is a data-driven reasoning technique. It starts with the known facts and applies the rules whose conditions are satisfied to derive new facts.
INITIAL FACTS
Fever = True
Cough = True
Breathlessness = True
INFERENCE STEP 1
Consider Rule 1:
F ∧ C → FL
We have:
F = True
C = True
Therefore, Rule 1 can be fired.
Fever AND Cough
↓
Possible Flu
New fact:
FL = True
INFERENCE STEP 2
Consider Rule 2:
F ∧ R → M
We have:
F = True
R = Unknown
The condition Rash is not available.
Therefore, Rule 2 cannot be fired.
Possible Measles = Not Derived
INFERENCE STEP 3
Consider Rule 3:
C ∧ B → P
We have:
C = True
B = True
Therefore, Rule 3 can be fired.
Cough AND Breathlessness
↓
Possible Pneumonia

New fact:
P = True
FORWARD CHAINING TABLE

Step	Rule	Conditions	Derived Fact
1	Rule 1	Fever = True, Cough = True	Possible Flu
2	Rule 2	Fever = True, Rash = Unknown	Not Derived
3	Rule 3	Cough = True, Breathlessness = True	Possible Pneumonia

FINAL DERIVED FACTS
Fever = True
Cough = True
Breathlessness = True
Possible Flu = True
Possible Pneumonia = True
Possible Measles is not derived because Rash is not given as a fact.

(c) BACKWARD CHAINING

GOAL
The required goal is:
Pneumonia(A)
Backward Chaining starts from the goal and works backward to find the rules and facts needed to prove the goal.

STEP 1 — FIND A RULE FOR THE GOAL
The rule that concludes Pneumonia is:
$C \land B \rightarrow P$
Therefore:
Pneumonia(A)
↓
Cough(A) AND Breathlessness(A)
The goal is reduced into two sub-goals:
Cough(A)
Breathlessness(A)

STEP 2 — VERIFY COUGH
Sub-goal:
Cough(A)
Given fact:
Cough(A) = True
Therefore:
Cough(A) ✓

STEP 3 — VERIFY BREATHLESSNESS
Sub-goal:
Breathlessness(A)
Given fact:
Breathlessness(A) = True
Therefore:
Breathlessness(A) ✓
Since both sub-goals are satisfied:
Cough(A) ∧ Breathlessness(A)

```
        ↓
    Pneumonia(A)
```
Therefore, the goal is proved.

GOAL REDUCTION TREE

```
 Pneumonia(A)
            |
            |
      Cough(A) AND Breathlessness(A)
         /          \
        /            \
       ✓              ✓
      Cough(A)       Breathlessness(A)
       |              |
       FACT            FACT
```

FINAL ANSWER

FORWARD CHAINING

Fever(A) AND Cough(A)
↓
Possible Flu(A)

Cough(A) AND Breathlessness(A)
↓
Possible Pneumonia(A)

Possible Measles is not derived because Rash is not known.

BACKWARD CHAINING

Pneumonia(A)
↓
Cough(A) AND Breathlessness(A)
↓
Both are given facts
↓
Pneumonia(A) is proved

CONCLUSION

The Smart Medical Diagnosis System uses propositional logic to represent patient symptoms and possible diagnoses. Forward Chaining derives **Possible Flu** and **Possible Pneumonia** from the given facts. Possible Measles cannot be derived because there is no Rash fact.
Using Backward Chaining, the goal **Pneumonia(A)** is reduced to **Cough(A)** and **Breathlessness(A)**. Since both facts are available in the knowledge base, **Patient A has Possible Pneumonia is successfully proved**.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

ANSWER:

A smart city traffic controller uses a First-Order Logic (FOL) knowledge base to manage emergency vehicle routing.

KNOWLEDGE BASE

RULE 1

∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)

RULE 2

∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x)

RULE 3

∀x∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)

GIVEN FACTS

Vehicle(Ambulance)
EmergencyType(Ambulance)
Vehicle(CarA)
Behind(CarA, Ambulance)

(a) UNIFICATION OF RULE 1 WITH GIVEN FACTS

STEP 1 — RULE 1

Rule 1 is:
Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)
The antecedent contains:
Vehicle(x)
EmergencyType(x)
The knowledge base contains:
Vehicle(Ambulance)
EmergencyType(Ambulance)
Therefore, we can substitute:
x = Ambulance
The substitution is:
θ_1 = {x/Ambulance}

STEP 2 — MATCH VEHICLE(x)

Rule:
Vehicle(x)
Fact:
Vehicle(Ambulance)
Unification gives:

x = Ambulance
Therefore:
θ_1 = {x/Ambulance}

STEP 3 — MATCH EMERGENCYTYPE(x)
Rule:
EmergencyType(x)
Fact:
EmergencyType(Ambulance)
Using the previous substitution:
x = Ambulance
Therefore:
θ_2 = {x/Ambulance}

STEP 4 — APPLY THE MGU
The Most General Unifier is:
θ = {x/Ambulance}
Substituting into Rule 1:
Vehicle(Ambulance) ∧ EmergencyType(Ambulance)
→ ClearPath(Ambulance)
Both facts are true.
Therefore:
ClearPath(Ambulance)
is derived.

UNIFICATION TABLE

Step	Rule Expression	Matching Fact	Substitution
1	Vehicle(x)	Vehicle(Ambulance)	{x/Ambulance}
2	EmergencyType(x)	EmergencyType(Ambulance)	{x/Ambulance}
3	Rule 1	Both conditions satisfied	θ = {x/Ambulance}

RESULT
θ = {x/Ambulance}
ClearPath(Ambulance)
is derived.

(b) RESOLUTION METHOD TO PROVE Proceed(CarA)
GOAL
We want to prove:
Proceed(CarA)
Resolution uses proof by contradiction.
Therefore, negate the goal:
¬Proceed(CarA)
We combine this negated goal with the relevant knowledge-base clauses and attempt to derive the empty clause:
□

STEP 1 — CONVERT RULE 1 INTO CNF
Original Rule 1:
Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)
Using:
P → Q ≡ ¬P ∨ Q

We get:
¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)
CNF Clause C1:
C1 = ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)

STEP 2 — CONVERT RULE 2 INTO CNF
Original Rule 2:
ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x)
This produces two clauses:
C2 = ¬ClearPath(x) ∨ GreenSignal(x)
C3 = ¬ClearPath(x) ∨ ¬RedSignal(x)

STEP 3 — CONVERT RULE 3 INTO CNF
Original Rule 3:
Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)
Therefore:
C4 = ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)

STEP 4 — GIVEN FACTS
C5 = Vehicle(Ambulance)
C6 = EmergencyType(Ambulance)
C7 = Vehicle(CarA)
C8 = Behind(CarA, Ambulance)

STEP 5 — RESOLUTION DERIVATION
From C1:
¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)
and:
Vehicle(Ambulance)
we substitute:
x = Ambulance
Resolving gives:
¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)
Using:
EmergencyType(Ambulance)
we obtain:
ClearPath(Ambulance)

STEP 6 — DERIVE GREEN SIGNAL
From C2:
¬ClearPath(x) ∨ GreenSignal(x)
and:
ClearPath(Ambulance)
substitute:
x = Ambulance
Therefore:
GreenSignal(Ambulance)
is derived.

STEP 7 — DERIVE Proceed(CarA)
C4 is:
¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)

Use:
Vehicle(CarA)
Behind(CarA, Ambulance)
GreenSignal(Ambulance)
Substitution:
x = CarA
y = Ambulance
The clause becomes:
¬Vehicle(CarA)
∨ ¬Behind(CarA,Ambulance)
∨ ¬GreenSignal(Ambulance)
∨ Proceed(CarA)
Resolving with the three facts eliminates the first three negative literals.
Therefore:
Proceed(CarA)
is derived.

STEP 8 — RESOLUTION WITH NEGATED GOAL
Negated goal:
¬Proceed(CarA)
We already derived:
Proceed(CarA)
Resolve:
Proceed(CarA)
with:
¬Proceed(CarA)
Therefore:
□
The empty clause is obtained.
Hence:
Proceed(CarA) is PROVED.

RESOLUTION PROOF TREE
Vehicle(Ambulance)
+
EmergencyType(Ambulance)
↓
ClearPath(Ambulance)
↓
GreenSignal(Ambulance)
+
Vehicle(CarA)
+
Behind(CarA,Ambulance)
↓
Proceed(CarA)
+
¬Proceed(CarA)
↓
□
Therefore:
Proceed(CarA) = TRUE

(c) TWO SIMULTANEOUS EMERGENCY VEHICLES ANALYSIS

The current knowledge base works for one emergency vehicle, such as:
Ambulance
because the following facts are available:
Vehicle(Ambulance)
EmergencyType(Ambulance)
From Rule 1:
Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)
we can derive:
ClearPath(Ambulance)
Then Rule 2 gives:
GreenSignal(Ambulance)
However, the knowledge base is not completely sufficient to handle two simultaneous emergency vehicles safely.

PROBLEM WITH TWO EMERGENCY VEHICLES

Suppose the second emergency vehicle is:
FireTruck
We need facts such as:
Vehicle(FireTruck)
EmergencyType(FireTruck)
Then Rule 1 can derive:
ClearPath(FireTruck)
and Rule 2 can derive:
GreenSignal(FireTruck)
However, the current KB does not specify:

- Which emergency vehicle has priority.
- How conflicting routes are handled.
- Whether two emergency vehicles can receive green signals simultaneously.
- How traffic signals should be coordinated.
- How to avoid collisions between emergency vehicles.
- Which vehicle should proceed first at an intersection.

Therefore, additional knowledge is required.

ADDITIONAL RULE 1 — EMERGENCY PRIORITY

A priority rule can be introduced:
∀x∀y:
EmergencyType(x) ∧ EmergencyType(y) ∧ HigherPriority(x,y)
→ PriorityVehicle(x)
This identifies which emergency vehicle should receive priority.

ADDITIONAL RULE 2 — SIGNAL CONTROL

A rule can specify that only the priority vehicle receives the required green signal:
∀x:
PriorityVehicle(x) → GreenSignal(x)

ADDITIONAL RULE 3 — CONFLICT AVOIDANCE

A rule can prevent two conflicting vehicles from proceeding simultaneously:
∀x∀y:
Proceed(x) ∧ ConflictingRoute(x,y)
→ ¬Proceed(y)

ADDITIONAL FACTS

For example:
Vehicle(FireTruck)
EmergencyType(FireTruck)
Behind(CarB, FireTruck)
HigherPriority(Ambulance, FireTruck)
These facts allow the system to reason about two emergency vehicles.

FINAL ANALYSIS
The existing FOL knowledge base can recognize multiple emergency vehicles if appropriate facts are added because the variables in the rules can be instantiated with different vehicle constants.
However, it does not contain sufficient rules for **priority management, route conflicts, collision avoidance, and simultaneous signal coordination**.
Therefore, additional rules and facts are required for safe multi-emergency-vehicle traffic management.

CONCLUSION
The Autonomous Traffic Management Agent demonstrates how First-Order Logic, Unification, and Resolution can be used for intelligent traffic management.
Unification of Rule 1 with the ambulance facts produces:
θ = {x/Ambulance}
This allows the system to derive:

ClearPath(Ambulance)
Resolution then derives:
GreenSignal(Ambulance)
and, using the facts about CarA:
Vehicle(CarA)
Behind(CarA,Ambulance)
the system proves:
Proceed(CarA)
by deriving the empty clause after adding the negated goal.
For two simultaneous emergency vehicles, the current knowledge base needs additional rules for **priority, conflict resolution, signal coordination, and collision avoidance** to ensure safe and logically consistent routing.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

ANSWER:

An agricultural advisory logical agent operates with the following propositional knowledge base (KB).

KNOWLEDGE BASE

P1: SoilDry → IrrigationNeeded

P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod

P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk

Facts:

SoilDry = True

CropWheat = True

(a) CONVERSION OF ALL SENTENCES INTO CNF

STEP 1 — CONVERT P1

Original statement:

SoilDry → IrrigationNeeded

Eliminate implication

Using:

P → Q ≡ ¬P ∨ Q

Therefore:

¬SoilDry ∨ IrrigationNeeded

CNF of P1

C1 = ¬SoilDry ∨ IrrigationNeeded

STEP 2 — CONVERT P2

Original statement:

IrrigationNeeded ∧ CropWheat → ApplyDripMethod

Eliminate implication

¬(IrrigationNeeded ∧ CropWheat) ∨ ApplyDripMethod

Using De Morgan's Law:

¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

CNF of P2

C2 = ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

This is already in CNF because it is a single disjunction of literals.

STEP 3 — CONVERT P3

Original statement:

¬ApplyDripMethod ∧ CropWheat → CropAtRisk

Eliminate implication

¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk

Apply De Morgan's Law

ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

CNF of P3

C3 = ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

STEP 4 — CONVERT THE FACTS

The first fact is:

SoilDry = True

Therefore:

C4 = SoilDry

The second fact is:

CropWheat = True

Therefore:

C5 = CropWheat

FINAL CNF KNOWLEDGE BASE

The complete CNF representation is:

C1 = ¬SoilDry ∨ IrrigationNeeded

C2 = ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

C3 = ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

C4 = SoilDry

C5 = CropWheat

(b) RESOLUTION REFUTATION

GOAL

We need to prove:

ApplyDripMethod

Resolution Refutation proves the goal by assuming its negation.

Therefore:

¬ApplyDripMethod

is added to the knowledge base.

NEGATED GOAL

C6 = ¬ApplyDripMethod

STEP 1 — DERIVE IRRIGATION NEEDED

Use:

C1 = ¬SoilDry ∨ IrrigationNeeded

and:

C4 = SoilDry
Resolve SoilDry and ¬SoilDry.
Therefore:
C7 = IrrigationNeeded
So:
SoilDry
↓
IrrigationNeeded

STEP 2 — DERIVE APPLY DRIP METHOD
Use:
C2 = ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod
We have:
C7 = IrrigationNeeded
and:
C5 = CropWheat
Resolve IrrigationNeeded:
¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod
with:
IrrigationNeeded
gives:
¬CropWheat ∨ ApplyDripMethod
Now resolve with:
CropWheat
Therefore:
C8 = ApplyDripMethod

STEP 3 — CONTRADICTION WITH NEGATED GOAL
We have derived:
C8 = ApplyDripMethod
The negated goal is:
C6 = ¬ApplyDripMethod
Resolve them:
ApplyDripMethod
+
¬ApplyDripMethod
↓
□
The empty clause □ is obtained.
Therefore:
ApplyDripMethod
is proved.

RESOLUTION PROOF TREE

SoilDry
|
|

¬SoilDry ∨ IrrigationNeeded
|
↓
IrrigationNeeded
|
|
¬IrrigationNeeded ∨ ¬CropWheat
∨ ApplyDripMethod
|
+
CropWheat
|
↓
ApplyDripMethod
|
+
¬ApplyDripMethod
(Negated Goal)
|
↓
□

Therefore:

ApplyDripMethod = TRUE

(c) REMOVE THE FACT SoilDry

Now remove:

SoilDry = True

from the knowledge base.

The remaining facts are:

CropWheat = True

The rules remain:

SoilDry → IrrigationNeeded

IrrigationNeeded ∧ CropWheat → ApplyDripMethod

¬ApplyDripMethod ∧ CropWheat → CropAtRisk

STEP 1 — CHECK IRRIGATION NEEDED

Rule P1:

SoilDry → IrrigationNeeded

To derive IrrigationNeeded, we need:

SoilDry = True

But the fact has been removed.

Therefore:

IrrigationNeeded = Not Derived

STEP 2 — CHECK APPLY DRIP METHOD

Rule P2:

IrrigationNeeded ∧ CropWheat → ApplyDripMethod

We have:

CropWheat = True

But:

IrrigationNeeded = Unknown

Therefore, both conditions are not satisfied.

So:

ApplyDripMethod = Not Derived

STEP 3 — CHECK CROP AT RISK

Rule P3:

¬ApplyDripMethod ∧ CropWheat → CropAtRisk

We know:

CropWheat = True

However, we do **not** know:

¬ApplyDripMethod = True

Not deriving ApplyDripMethod does **not** mean that ¬ApplyDripMethod is true.

This is an important principle of logical reasoning:

Not Proven ≠ False

Therefore:

CropAtRisk = Not Derived

LOGICAL CONCLUSION AFTER REMOVING SoilDry

Fact/Conclusion	Status
SoilDry	Removed
CropWheat	True
IrrigationNeeded	Not Derived
ApplyDripMethod	Not Derived
CropAtRisk	Not Derived

IMPORTANT LOGICAL POINT

The removal of SoilDry does **not** allow us to conclude:

¬SoilDry

Similarly, failure to prove:

ApplyDripMethod

does not allow us to conclude:

¬ApplyDripMethod

Therefore, CropAtRisk cannot be logically derived.

FINAL ANSWER

(a) CNF

C1 = ¬SoilDry ∨ IrrigationNeeded

C2 = ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

C3 = ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

C4 = SoilDry

C5 = CropWheat

(b) Resolution Refutation

SoilDry

↓

IrrigationNeeded

↓

ApplyDripMethod

↓

¬ApplyDripMethod

↓

□

Therefore:

ApplyDripMethod is proved.

(c) After Removing SoilDry

SoilDry → IrrigationNeeded

cannot be applied.

Therefore:

IrrigationNeeded = Not Derived

Hence:

ApplyDripMethod = Not Derived

and because ¬ApplyDripMethod is also not given:

CropAtRisk = Not Derived

CONCLUSION

The Agricultural Expert Reasoning System uses CNF and Resolution to derive logical conclusions. With SoilDry and CropWheat as facts, the system proves ApplyDripMethod through Resolution Refutation. If SoilDry is removed, the reasoning chain stops at IrrigationNeeded, so ApplyDripMethod cannot be proved. CropAtRisk also does not follow because the absence of proof for ApplyDripMethod is not equivalent to proving ¬ApplyDripMethod.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

ANSWER:

An AI agent is navigating the Wumpus World. It receives the following percepts:
Stench at [1,2]
Breeze at [1,1]
Glitter at [2,2]
The agent uses a propositional knowledge base to reason about the environment.

KNOWLEDGE BASE

Rule 1
Stench[1,2] → WumpusAdjacent[1,2]
Rule 2
Breeze[1,1] → PitAdjacent[1,1]
Rule 3
Glitter[x,y] → Gold[x,y]
Rule 4
¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) CONSTRUCT THE AGENT'S BELIEF STATE

INITIAL PERCEPTS

The agent initially perceives:
Stench[1,2] = True
Breeze[1,1] = True
Glitter[2,2] = True
Therefore, the initial belief state is:
B = {
   Stench[1,2],
   Breeze[1,1],
   Glitter[2,2]
}

APPLY MODUS PONENS

Modus Ponens has the form:
P → Q
P

Q
If the condition of a rule is true, its conclusion can be derived.

INFERENCE STEP 1 — STENCH
Given Rule 1:
Stench[1,2] → WumpusAdjacent[1,2]
Fact:
Stench[1,2]
Therefore, by Modus Ponens:
Stench[1,2]
↓
WumpusAdjacent[1,2]
Derived fact:
WumpusAdjacent[1,2] = True
This means the Wumpus is in one of the cells adjacent to [1,2].

INFERENCE STEP 2 — BREEZE
Given Rule 2:
Breeze[1,1] → PitAdjacent[1,1]
Fact:
Breeze[1,1]
Therefore:
Breeze[1,1]
↓
PitAdjacent[1,1]
Derived fact:
PitAdjacent[1,1] = True
This means there may be a pit in a cell adjacent to [1,1].

INFERENCE STEP 3 — GLITTER
Given Rule 3:
Glitter[x,y] → Gold[x,y]
We have:
Glitter[2,2]
Substitute:
x = 2
y = 2
Therefore:
Glitter[2,2]
↓
Gold[2,2]
Derived fact:
Gold[2,2] = True

INFERENCE STEP 4 — SAFE CELL
Rule 4 is:
¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]
To use this rule, the agent must know both:
¬Wumpus[x,y]
and
¬Pit[x,y]
These facts have not been provided.
Therefore, Rule 4 cannot currently be fired.
Safe[x,y] = Not Derived

BELIEF STATE AFTER REASONING
The agent's knowledge becomes:
Stench[1,2]
Breeze[1,1]
Glitter[2,2]

WumpusAdjacent[1,2]
PitAdjacent[1,1]
Gold[2,2]

DERIVED CONCLUSIONS

Percept	Rule Applied	Derived Knowledge
Stench[1,2]	Rule 1	WumpusAdjacent[1,2]
Breeze[1,1]	Rule 2	PitAdjacent[1,1]
Glitter[2,2]	Rule 3	Gold[2,2]
No known ¬Wumpus and ¬Pit	Rule 4	Safe cannot be derived

(b) FORWARD CHAINING
Forward Chaining starts from the available percepts and repeatedly applies rules to generate new knowledge.
STEP 1
Initial fact:
Stench[1,2]
Apply Rule 1:
Stench[1,2] → WumpusAdjacent[1,2]
Therefore:
WumpusAdjacent[1,2]
is derived.

POSSIBLE WUMPUS LOCATIONS
In a standard Wumpus World grid, the cells directly adjacent to [1,2] are:
[1,1]
[1,3]
[2,2]
Therefore:
Possible Wumpus locations:
[1,1], [1,3], [2,2]
However, the statement WumpusAdjacent[1,2] only tells us that the Wumpus is adjacent to [1,2]; it does **not** identify one exact location.

STEP 2 — BREEZE
Initial fact:
Breeze[1,1]
Apply Rule 2:
Breeze[1,1] → PitAdjacent[1,1]
Therefore:
PitAdjacent[1,1]
is derived.

POSSIBLE PIT LOCATIONS
The cells adjacent to [1,1] in a standard grid are:

[1,2]
[2,1]
Therefore:
Possible Pit locations:
[1,2], [2,1]
Again, the knowledge only identifies possible locations, not the exact pit location.

STEP 3 — GLITTER
Initial fact:
Glitter[2,2]
Apply Rule 3:
Glitter[2,2] → Gold[2,2]
Therefore:
Gold[2,2]
is derived.

FORWARD CHAINING SUMMARY
Stench[1,2]
↓
WumpusAdjacent[1,2]
↓
Possible Wumpus:
[1,1], [1,3], [2,2]
Breeze[1,1]
↓
PitAdjacent[1,1]
↓
Possible Pit:
[1,2], [2,1]
Glitter[2,2]
↓
Gold[2,2]

FORWARD CHAINING TABLE

Step	Current Fact	Rule Applied	New Knowledge
1	Stench[1,2]	Rule 1	WumpusAdjacent[1,2]
2	Breeze[1,1]	Rule 2	PitAdjacent[1,1]
3	Glitter[2,2]	Rule 3	Gold[2,2]
4	¬Wumpus and ¬Pit not known	Rule 4	Safe not derived

(c) SAFETY OF THE PATH
The proposed path is:
[1,1] → [1,2] → [2,2]
The objective is to reach [2,2] and collect the gold.
From the percepts, we know:
Breeze[1,1]
Stench[1,2]
Glitter[2,2]

CHECK CELL [1,1]
The agent starts at:

[1,1]
A Breeze is perceived at [1,1].
Therefore:
PitAdjacent[1,1]
is known.
The possible pit locations are:
[1,2]
[2,1]
Therefore, [1,2] may contain a pit.
So the movement:
[1,1] → [1,2]
cannot be guaranteed to be safe.

CHECK CELL [1,2]
At [1,2], there is a Stench:
Stench[1,2]
Therefore:
WumpusAdjacent[1,2]
Possible Wumpus locations include:
[1,1]
[1,3]
[2,2]
Therefore, [2,2] is a possible Wumpus location.
This creates a significant danger because [2,2] is also the location of the gold.

CHECK CELL [2,2]
From the Glitter percept:
Glitter[2,2]
we can confidently derive:
Gold[2,2]
However, we cannot derive:
¬Wumpus[2,2]
and:
¬Pit[2,2]
Therefore, Rule 4 cannot establish:
Safe[2,2]

PATH ANALYSIS
Proposed path:
[1,1] → [1,2] → [2,2]
First movement
[1,1] → [1,2]
Potential danger:
Pit[1,2]
because Breeze[1,1] indicates a possible pit in an adjacent cell.
Second movement
[1,2] → [2,2]
Potential danger:
Wumpus[2,2]
because Stench[1,2] indicates that the Wumpus may be adjacent to [1,2].

SAFETY CONCLUSION
The path:

[1,1] → [1,2] → [2,2]
cannot be confirmed as safe using the current knowledge base.
The agent knows:
Gold[2,2] = True
but does not know:
¬Wumpus[2,2]
¬Pit[2,2]
Therefore, Rule 4 cannot prove:
Safe[2,2]
Also, [1,2] is a possible pit location and [2,2] is a possible Wumpus location.

FINAL ANSWER
(a) BELIEF STATE
Initial Percepts:
Stench[1,2]
Breeze[1,1]
Glitter[2,2]
Derived:
WumpusAdjacent[1,2]
PitAdjacent[1,1]
Gold[2,2]

(b) POSSIBLE LOCATIONS
Wumpus
From:
Stench[1,2]
Possible locations:
[1,1], [1,3], [2,2]
Pit
From:
Breeze[1,1]
Possible locations:
[1,2], [2,1]
Gold
From:
Glitter[2,2]
we know:
Gold[2,2]

(c) PATH SAFETY
[1,1] → [1,2] → [2,2]
The path is **NOT GUARANTEED TO BE SAFE**.
Reason:
Breeze[1,1]
↓
Possible Pit[1,2]
and:
Stench[1,2]
↓
Possible Wumpus[2,2]
Therefore, the agent should not classify the entire path as safe until additional information is obtained.

CONCLUSION
The Wumpus World agent uses percepts, Modus Ponens, and Forward Chaining to build its belief state. It can identify the **possible locations** of hazards and the exact location of the gold, but the current knowledge is insufficient to prove the proposed path safe. A rational agent should therefore gather additional information or choose a path whose cells can be logically established as safe.